

LABSHEET - 7**HIMANI CHAUDHARY****BTECH(CSE) 3RD YEAR****CU240250967****SECTION - B****ROLL NO. - 24****COMPUTER VISION**

```
import cv2
import numpy as np
import os
from google.colab.patches import cv2_imshow
from IPython.display import clear_output

# =====
# EXPERIMENT NO. 7
# Motion Estimation using Optical Flow Algorithms
# Lucas-Kanade Sparse + Farneback Dense Optical Flow
# =====

# =====
# VIDEO PATH
# =====

VIDEO_PATH = "/content/WhatsApp Video 2026-09-10 at 9.28.37 AM.mp4"

# =====
# CHECK VIDEO FILE
# =====

if not os.path.exists(VIDEO_PATH):

    print("=====")
    print("ERROR: VIDEO FILE NOT FOUND")
    print("=====")
    print("Expected file:")
    print(VIDEO_PATH)
    print()
    print("Please upload the MP4 file to Google Colab.")
    print("Use the Files icon on the left side of Colab.")

else:

    print("Video file found successfully.")
```

```

# =====
# OPEN VIDEO
# =====

cap = cv2.VideoCapture(VIDEO_PATH)

if not cap.isOpened():

    print("=====")
    print("ERROR: COULD NOT OPEN VIDEO")
    print("=====")
    raise SystemExit

# =====
# READ FIRST FRAME
# =====

ret, old_frame = cap.read()

if not ret or old_frame is None:

    print("=====")
    print("ERROR: COULD NOT READ FIRST FRAME")
    print("=====")

    cap.release()
    raise SystemExit

# =====
# CONVERT FIRST FRAME TO GRAYSCALE
# =====

old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

# =====
# LUCAS-KANADE PARAMETERS
# =====

feature_params = {
    "maxCorners": 100,
    "qualityLevel": 0.3,
    "minDistance": 7,
    "blockSize": 7
}

lk_params = {

```

```

        "winSize": (15, 15),
        "maxLevel": 2,
        "criteria": (
            cv2.TERM_CRITERIA_EPS |
            cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    }

# =====
# FIND INITIAL FEATURES
# =====

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

# =====
# TRAJECTORY MASK
# =====

mask = np.zeros_like(old_frame)

# =====
# VIDEO INFORMATION
# =====

fps = cap.get(cv2.CAP_PROP_FPS)

if fps <= 0:
    fps = 30

total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)

# =====
# PROGRAM INFORMATION
# =====

print()
print("=====")
print("          EXPERIMENT NO. 7")
print("=====")
print("Motion Estimation using Optical Flow")
print("-----")
print("Lucas-Kanade : Sparse Optical Flow")
print("Farneback      : Dense Optical Flow")

```

```

print("-----")
print("Student Name : HIMANI CHAUDHARY")
print("Roll Number : CU240250967")
print("-----")
print("Total Frames :", total_frames)
print("FPS          :", round(fps, 2))
print("=====")
print()
print("Processing video...")
print()

# =====
# FRAME COUNTER
# =====

frame_count = 0

# =====
# PROCESS VIDEO
# =====

while True:

    # -----
    # READ FRAME
    # -----

    ret, frame = cap.read()

    if not ret:
        break

    frame_count += 1

    # -----
    # GRAYSCALE
    # -----

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # =====
    # LUCAS-KANADE SPARSE OPTICAL FLOW
    # =====

    sparse_frame = frame.copy()

```

```
if p0 is not None and len(p0) > 0:

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is not None and status is not None:

        good_new = p1[status.ravel() == 1]
        good_old = p0[status.ravel() == 1]

        # -----
        # DRAW TRACKED POINTS
        # -----

        for new, old in zip(
            good_new,
            good_old
        ):

            a, b = new.ravel()
            c, d = old.ravel()

            a = int(a)
            b = int(b)
            c = int(c)
            d = int(d)

            # Draw trajectory
            mask = cv2.line(
                mask,
                (a, b),
                (c, d),
                (0, 255, 0),
                2
            )

            # Draw feature point
            sparse_frame = cv2.circle(
                sparse_frame,
                (a, b),
                5,
                (0, 0, 255),
                -1
            )
```

```
# -----  
# COMBINE OUTPUT  
# -----  
  
sparse_output = cv2.add(  
    sparse_frame,  
    mask  
)  
  
# -----  
# UPDATE POINTS  
# -----  
  
if len(good_new) > 0:  
  
    p0 = good_new.reshape(  
        -1,  
        1,  
        2  
    )  
  
else:  
  
    p0 = cv2.goodFeaturesToTrack(  
        frame_gray,  
        mask=None,  
        **feature_params  
    )  
  
    mask = np.zeros_like(frame)  
  
else:  
  
    sparse_output = sparse_frame  
  
    p0 = cv2.goodFeaturesToTrack(  
        frame_gray,  
        mask=None,  
        **feature_params  
    )  
  
    mask = np.zeros_like(frame)  
  
else:  
  
    sparse_output = sparse_frame  
  
    p0 = cv2.goodFeaturesToTrack(  
        frame_gray,  
        mask=None,  
        **feature_params
```

```
)

mask = np.zeros_like(frame)

# =====
# FARNEBACK DENSE OPTICAL FLOW
# =====

flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# =====
# MAGNITUDE AND ANGLE
# =====

magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# =====
# HSV VISUALIZATION
# =====

hsv = np.zeros_like(
    cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2HSV
    )
)

hsv[..., 1] = 255

hsv[..., 0] = (
    angle * 180 / np.pi / 2
).astype(np.uint8)

hsv[..., 2] = cv2.normalize(
    magnitude,
```

```

        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)

# =====
# CONVERT HSV TO BGR
# =====

dense_output = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

# =====
# ADD TEXT
# =====

cv2.putText(
    frame,
    "Original Video",
    (10, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.9,
    (0, 255, 255),
    2
)

cv2.putText(
    sparse_output,
    "Lucas-Kanade Sparse Optical Flow",
    (10, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 255),
    2
)

cv2.putText(
    dense_output,
    "Farneback Dense Optical Flow",
    (10, 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (255, 255, 255),
    2
)

# =====

```

```

# DISPLAY IN GOOGLE COLAB
# =====

# Display every 10th frame.
# This prevents Colab from becoming too slow.

if frame_count % 10 == 0:

    clear_output(wait=True)

    print("=====")
    print("      EXPERIMENT NO. 7 - OUTPUT")
    print("=====")
    print("Student Name : HIMANI CHAUDHARY")
    print("Roll Number  : CU240250967")
    print("Frame       :", frame_count)
    print("Total Frames :", total_frames)
    print("-----")

    print("\n1. ORIGINAL VIDEO")
    cv2_imshow(frame)

    print("\n2. LUCAS-KANADE SPARSE OPTICAL FLOW")
    cv2_imshow(sparse_output)

    print("\n3. FARNEBACK DENSE OPTICAL FLOW")
    cv2_imshow(dense_output)

# =====
# UPDATE PREVIOUS FRAME
# =====

old_gray = frame_gray.copy()

# =====
# RELEASE VIDEO
# =====

cap.release()

# =====
# FINAL OUTPUT
# =====

print()
print("=====")
print("  PROGRAM COMPLETED SUCCESSFULLY")
print("=====")
print("Experiment No. : 7")
print("Student Name   : HIMANI CHAUDHARY")
print("Roll Number    : CU240250967")
print("Frames Processed:", frame_count)

```

```
print("=====")
```



=====

EXPERIMENT NO. 7 - OUTPUT

=====

Student Name : HIMANI CHAUDHARY

Roll Number : CU240250967

Frame : 20

Total Frames : 23

1. ORIGINAL VIDEO



2. LUCAS-KANADE SPARSE OPTICAL FLOW

